



양준영

Backend Engineer.

반복적인 프로세스 개선과 정교한 기능 설계에 대한 관심이 많습니다.

AI Agent를 활용하며 자동화된 워크플로우를 구축하고 기획부터 개발까지 진행하며 시간과 비용을 절약해 본 경험이 있습니다. 결제 기능의 결함을 찾아내고 기능의 안정성을 개선해본 경험이 있습니다. AI를 활용한 빠른 생산성으로 일을 효율적으로 처리하며 비즈니스 이해를 바탕으로 기능 안정성까지 고민하는 개발자가 되기 위해 노력하고 있습니다.

About



1998. 07. 22



sgo722@naver.com



<https://github.com/sgo722>

Study

✦ 객체지향 미션 진행 2025.03 ~ 2025.05 github.com/OOPMISSION

✦ Real My SQL 1, 2권 2025.01 ~ 2025.03 notion.com/RealMySQL

Experience



세종대학교 컴퓨터공학과 2022.03 ~ 2024.02



아름 세종대학교 앱개발 동아리 2023.03 ~ 2024.02



삼성청년SW 아카데미 11기 2024.01 ~ 2024.12

서비스 소개

'코드엘'은 가치관 기반 소셜 매칭 서비스로, 프로필 기반 매칭, 실시간 채팅, 추천 시스템을 제공합니다. 현재 400명 이상의 실사용자가 이용 중인 실서비스입니다.

기술 스택

- ◆ Kotlin, SpringBoot, MySQL, AWS(ECS), Redis, WebSocket, k6, RDS
- ◆ Grafana, Prothemeus

담당 기능

- WebSocket/STOMP 기반 실시간 채팅 시스템 구현
- k6 활용 WebSocket 부하테스트 환경 구축
- 전략 패턴 기반 하위호환성 API 설계
- AI 도구(Claude Code) 활용한 개발 자동화
- Thymeleaf 기반 관리자 페이지 개발

고민했던 점

1. AI Agent 기반 개발 [원문보기](#)

- 백엔드를 혼자 담당하면서 컨텍스트 스위칭과 반복 작업 생산성을 저하시키는 문제 인지
- tmux + git worktree로 병렬 개발 환경 구축, 프롬프트 엔지니어링으로 AI 응답 정확도 향상
- 워크플로우 템플릿화와 GitHub Actions + AI 테스트 루프로 자동화 구축
- 컨텍스트 스위칭 0초, PR 생성 80% 단축, 테스트 디버깅 80% 단축 달성
- 단순 도구 활용이 아닌, 인프라 → 방법론 → 자동화 순서로 체계적 접근하여 혼자서도 팀 수준의 생산성 확보

2. 단일 서버에서 다중 서버로의 전환 [원문보기](#)

- 회원 200명 도달 시점에 단일 서버의 근본적 한계(인스턴스 장애 시 전체 중단, 수평 확장 불가) 인지
- SimpleBroker 인메모리 방식으로 다중 서버 간 WebSocket 메시지 미전달, synchronized의 JVM 독립 동작으로 동시성 제어 불가, 스케줄러 중복 실행 문제 발생
- Redis Pub/Sub + SimpleBroker로 서버 간 메시지 중계, Redisson 분산락으로 동시성 제어, ShedLock으로 스케줄러 중복 방지
- 로컬 전송 우선 구조로 Redis 장애 시에도 같은 서버 사용자는 정상 동작하도록 안정성 확보
- 단일 장애점 제거, 수평 확장 구조로 회원 400명 → 1,000명+ 대응 가능

서비스 소개

'터득'은 조건에 따라 선정된 회원에게 보상을 지급하는 클릭 게임으로, 사용자에게 재미와 보상을 제공하는 서비스입니다.

기술 스택

- ◆ Java, SpringBoot, MySQL, AWS(ECS), Redis, Kafka
- ◆ Socket, Grafana, Prothemeus, nGrinder

담당 기능

- 카카오페이 외부 API 트랜잭션 분리
- 클릭 기능 동시성 문제 해결
- 아키텍처 확장을 통한 성능 개선
- 슬랙, 디스코드 알람 기능 구현
- 각 아키텍처 버전 부하테스트 비교

고민했던 점

1. 결제 API 트랜잭션 분리 [원문보기](#) [요약보기](#)

- 외부 결제 API와 DB 반영을 하나의 트랜잭션으로 처리할 경우 발생하는 **정합성 문제와 커넥션 점유 이슈 인지**
- 결제 승인과 아이템 지급 로직을 **트랜잭션 분리**하고, **스케줄링 + 재시도 + 알람 시스템**을 도입해 자동/수동 복구 구조 구성
- **RestTemplate timeout, 트랜잭션 timeout** 설정을 통해 커넥션 점유 시간 최소화
- 사용자 경험 보장을 위해 재시도 로직 3회 적용, 실패 시 Slack 알람 연동하여 개발자 수동 개입 가능하도록 설계
- 기능 완성도 향상을 위해 JPA 쓰기 지연 예외, 동시성 문제, 가독성 개선까지 고려하며 설계 안정성 확보

2. 클릭 기능 동시성 문제 해결 [원문보기](#) [요약보기](#)

- 동시에 다수 사용자가 클릭하는 환경에서 **클릭 수 불일치 및 보상자 중복 문제** 발생
- 단순한 증가 연산이라는 기능적 특성에 기반해, 여러 동시성 처리 기법(synchronized, 락, 분산락 등)을 비교 분석
- 최종적으로 **Redis INCR의 원자 연산**이 성능 및 구조 양면에서 가장 적합하다고 판단
- TPS 약 4배 향상, 분산 환경에서도 안정적인 보상 로직 구현
- 기능 이해 기반 기술 적용으로, 과도한 설계나 **불필요한 오버엔지니어링을 지양하며 실용적인 개선** 달성

3. 아키텍처 확장에 따른 성능 개선 [원문보기](#) [요약보기](#)

- 클릭 기반 고부하 기능에서 성능 병목을 분석해 **동시성 처리, 로그 저장, 리소스 풀 튜닝** 단계로 나누어 개선 진행
- 총 클릭 수는 RDB → Redis INCR로 분리, 로그 저장은 비동기 처리
- 이후 Thread/Connection Pool 튜닝을 통해 **최종 TPS를 약 6.5배**까지 개선
- 단순 기술 도입이 아닌, 구조적 병목 원인 분석과 상황 맞춤 기술 선택을 통해 실질적인 성능 향상 달성

서비스 소개

수많은 프로젝트들이 복잡한 배포 과정으로 인해 빛을 보지 못하는 현실을 목격하고 시작하게 되었습니다. 노트북 하나만 있으면 누구나 몇 번의 클릭으로 자신의 프로젝트를 손쉽게 배포할 수 있는 서비스 입니다.

기술 스택

- ◆ Java, SpringBoot, PostgreSQL, AWS(ECS), Redis, Kafka
- ◆ PostgreSQL, AWS(ECS), Kafka

담당 기능

- 도커파일 제작 기능 구현
- Saga 코레오그래피 패턴 적용
- MSA 환경에서 Deployment 도메인 개발

고민했던 점

1. 도커 파일 제작 기능 구현 [원문보기](#)

- Git 레포지토리에 Dockerfile이 없는 경우, **AI를 활용해 Dockerfile을 자동 생성**하여 배포
- 초기에는 빌드도구(FE/BE 구분)만을 기준으로 AI에 전달하여 생성 → 정보 부족으로 **성공률 20%**
- 이후 언어, 빌드도구, 패키지 관리자 정보까지 함께 전달하여 AI 생성 정확도 일부 향상 → **성공률 50%**
- Java단에서 수집된 프로젝트 정보를 기반으로 분기 처리하여 직접 Dockerfile 생성 로직 구현 → **성공률 90%**
- AI 의존도를 낮추고, **배포 성공률 향상 및 AI 크레딧 소모 절감 효과** 달성

2. SAGA 패턴 적용 [원문보기](#)

- Project와 Deployment가 도메인 분리된 상태에서 1:N 관계로 운영되고 있었음
- Project 삭제 시 Deployment가 삭제되지 않아 **데이터 불일치 문제**가 발생
- 양쪽 도메인에 status 필드를 추가해 **Soft Delete 방식**으로 전환
- Deployment 삭제 실패 시 Project 삭제도 롤백되도록 **코레오그래피 방식**의 보상 트랜잭션 적용
- 서비스 간 정합성을 유지하면서 도메인 결합도는 낮추는 방향으로 개선함